Laravel Routing — Complete Teaching Documentation

Laravel 12 • PHP Web Development • Beginner to Intermediate

This documentation covers Laravel routing from the fundamentals through practical application development, including route definitions, HTTP methods, parameters, named routes, route groups, middleware, resource routes, redirects, validation, and route debugging.

All examples use Laravel 12 and the standard routes/web.php file unless stated otherwise.

1. What is routing?

Routing is the process of determining how an application responds to an incoming HTTP request.

When a user visits a URL, Laravel checks the request method and URI, finds a matching route, and executes its assigned action.

For example:

```
Route::get('/hello', function () {
    return 'Hello, Laravel!';
});
```

When the user visits /hello using a GET request, Laravel returns the text Hello, Laravel!.

A route typically contains:

- HTTP method — GET, POST, PUT, PATCH, DELETE, etc.
- URI — the path requested by the user.
- Action — the function or controller method that handles the request.
- Optional name, middleware, parameter constraints, and other configuration.

2. Where are routes defined?

Laravel stores its route definitions in the routes directory.

File	Purpose
routes/web.php	Browser-based web routes with session and CSRF support
routes/api.php	API routes, when the API routing file is installed
routes/console.php	Defines custom Artisan console commands
routes/channels.php	Defines broadcasting channels, when applicable

For most beginner web applications, we will work with routes/web.php.

A newly created Laravel project may not include every optional route file. API routing can be installed using Laravel's API installation command.

2.1 Basic route syntax

```
use Illuminate\Support\Facades\Route;

Route::get('/welcome', function () {
    return 'Welcome to Laravel';
});
```

Explanation:

- Route is Laravel's routing facade.
- get() defines a route that responds to GET requests.
- /welcome is the URI.
- The anonymous function is the route action.
- return sends the response back to the browser.

2.2 Route responses

A route can return different types of responses.

Plain text:

```
Route::get('/text', function () {
    return 'Hello World';
});
```

HTML:

```
Route::get('/html', function () {
    return '<h1>Hello World</h1>';
});
```

JSON:

```
Route::get('/json', function () {
    return response()->json([
        'name' => 'Rahul',
        'course' => 'PHP',
    ]);
});
```

Blade view:

```
Route::get('/home', function () {
    return view('home');
});
```

The last example expects a file at resources/views/home.blade.php.

3. HTTP methods in Laravel

HTTP methods specify what kind of operation a client wants to perform.

3.1 GET route

GET is primarily used to retrieve or display information.

```
Route::get('/students', function () {
    return 'All students';
});
```

Examples of GET requests:

- Displaying a student list.
- Viewing a product.
- Opening a registration form.
- Searching for information.

3.2 POST route

POST is commonly used to submit data to the server.

```
Route::post('/students', function () {
    return 'Student data submitted';
});
```

POST requests are usually sent through an HTML form or an API client.

3.3 PUT and PATCH routes

PUT and PATCH are commonly used to update existing resources.

```
Route::put('/students/{id}', function ($id) {
    return "Replace student $id";
});

Route::patch('/students/{id}', function ($id) {
    return "Update student $id";
});
```

A PUT request generally represents replacing a resource, while PATCH represents a partial update. The exact behavior depends on how the application implements the operation.

3.4 DELETE route

DELETE is used to request deletion of a resource.

```
Route::delete('/students/{id}', function ($id) {
    return "Delete student $id";
});
```

3.5 Route::any()

Laravel also provides any() for routes that should accept all supported HTTP methods.

```
Route::any('/test', function () {
    return 'Request received';
});
```

Use this sparingly. In most applications, defining the specific HTTP method is clearer and safer.

4. Route parameters

Route parameters allow dynamic values to be passed through the URI.

For example, a single route can display many student profiles.

4.1 Required parameters

```
Route::get('/student/{id}', function ($id) {
    return "Student ID: $id";
});
```

URL	Result
/student/1	Student ID: 1
/student/25	Student ID: 25
/student/100	Student ID: 100

The value in {id} is passed to the callback as $id.

4.2 Multiple parameters

```
Route::get('/student/{id}/{name}', function ($id, $name) {
    return "ID: $id, Name: $name";
});
```

URL:

```
/student/10/Rahul
```

Output:

```
ID: 10, Name: Rahul
```

The parameter names in the URI and the callback must correspond, and the callback arguments must follow the URI parameter order.

4.3 Optional parameters

Use ? to make a route parameter optional.

```
Route::get('/greet/{name?}', function ($name = 'Guest') {
    return "Welcome, $name";
});
```

- /greet returns Welcome, Guest.
- /greet/Rahul returns Welcome, Rahul.

The callback needs a default value for an optional parameter.

4.4 Parameter constraints

Laravel allows you to restrict which values match a route.

```
Route::get('/student/{id}', function ($id) {
    return "Student ID: $id";
})->whereNumber('id');
```

This route matches /student/10, but not /student/abc.

Other useful constraints:

```
// Alphabetic characters only
Route::get('/user/{name}', function ($name) {
    return $name;
})->whereAlpha('name');

// Alphanumeric characters only
Route::get('/code/{code}', function ($code) {
    return $code;
})->whereAlphaNumeric('code');

// Regular expression
Route::get('/year/{year}', function ($year) {
    return $year;
})->where('year', '[0-9]{4}');
```

Constraints are useful for preventing invalid URL values from matching a route.

5. Named routes

Named routes allow you to identify routes using a unique name rather than repeatedly writing their URI.

5.1 Defining a named route

```
Route::get('/about', function () {
    return view('about');
})->name('about');
```

The route name is about.

5.2 Generating URLs

```
$url = route('about');
```

This generates the URL associated with the named route.

In a Blade template:

```
<a href="{{ route('about') }}">About Us</a>
```

If the route URI changes, Laravel can generate the updated URL automatically.

5.3 Named routes with parameters

```
Route::get('/student/{id}', function ($id) {
    return "Student ID: $id";
})->name('student.show');
```

Generate the URL:

```
$url = route('student.show', ['id' => 10]);
```

Result:

```
/student/10
```

5.4 Redirecting to a named route

```
Route::get('/old-about', function () {
    return redirect()->route('about');
});
```

This redirects the user to the route named about.

6. Route groups

Route groups allow you to apply shared configuration to multiple routes.

6.1 Prefix groups

```
Route::prefix('admin')->group(function () {

    Route::get('/dashboard', function () {
        return 'Admin Dashboard';
    });

    Route::get('/users', function () {
        return 'Manage Users';
    });

});
```

The resulting URLs are:

- /admin/dashboard
- /admin/users

6.2 Name prefix groups

```
Route::name('admin.')->group(function () {

    Route::get('/dashboard', function () {
        return 'Dashboard';
    })->name('dashboard');

    Route::get('/users', function () {
        return 'Users';
    })->name('users');

});
```

The route names are:

- admin.dashboard
- admin.users

6.3 Combining prefix and name groups

```
Route::prefix('admin')
    ->name('admin.')
    ->group(function () {

        Route::get('/dashboard', function () {
            return 'Dashboard';
        })->name('dashboard');

    });
```

This creates the URI /admin/dashboard and the route name admin.dashboard.

7. Middleware in routes

Middleware filters or processes HTTP requests before they reach the route action.

Common uses include authentication, authorization, request logging, and rate limiting.

7.1 Applying middleware

```
Route::get('/dashboard', function () {
    return 'Dashboard';
})->middleware('auth');
```

Only authenticated users can access this route.

7.2 Middleware groups

```
Route::middleware('auth')->group(function () {

    Route::get('/profile', function () {
        return 'User Profile';
    });

    Route::get('/settings', function () {
        return 'User Settings';
    });

});
```

Both routes require authentication.

7.3 Route groups with prefixes and middleware

```
Route::prefix('admin')
    ->middleware('auth')
    ->group(function () {

        Route::get('/dashboard', function () {
            return 'Admin Dashboard';
        });

        Route::get('/reports', function () {
            return 'Reports';
        });

    });
```

Both routes require authentication and use the /admin prefix.

Important: Authentication alone does not guarantee administrator access. Use appropriate authorization middleware or policies to restrict administrator-only operations.

8. Resource routes

Laravel provides resource routing for standard CRUD operations.

CRUD stands for Create, Read, Update, and Delete.

A resource controller commonly handles:

- Displaying a list.
- Displaying a form to create a record.
- Saving a new record.
- Displaying one record.
- Displaying an edit form.
- Updating a record.
- Deleting a record.

8.1 Creating a resource controller

```
php artisan make:controller StudentController --resource
```

This creates a controller with the standard resource methods.

8.2 Defining a resource route

```
Route::resource('students', StudentController::class);
```

Laravel automatically registers the standard resource routes.

Method	URI	Controller method
GET	/students	index
GET	/students/create	create
POST	/students	store
GET	/students/{student}	show
GET	/students/{student}/edit	edit
PUT/ PATCH	/students/{student}	update
DELETE	/students/{student}	destroy

You do not need to manually define each of these routes.

8.3 Limiting resource routes

```
Route::resource('students', StudentController::class)
    ->only(['index', 'show']);
```

This registers only the listing and detail routes.

Alternatively:

```
Route::resource('students', StudentController::class)
    ->except(['destroy']);
```

This registers all standard resource routes except deletion.

9. Form submission and CSRF protection

Laravel's web routes use CSRF protection for state-changing requests.

A typical form:

```
<form action="{{ route('students.store') }}" method="POST">
    @csrf

    <input type="text" name="name" required>

    <button type="submit">Save</button>
</form>
```

The @csrf directive generates a hidden token field. Laravel verifies the token when the form is submitted.

For PUT, PATCH, and DELETE requests, HTML forms use POST with a method override.

```
<form action="{{ route('students.update', 1) }}" method="POST">
    @csrf
    @method('PUT')

    <input type="text" name="name">

    <button type="submit">Update</button>
</form>
```

Laravel reads the hidden _method field and treats the request as PUT.

10. Route validation and request handling

Routes can receive user input, but input should be validated before it is used.

Example:

```
use Illuminate\Http\Request;

Route::post('/students', function (Request $request) {

    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email',
    ]);

    return response()->json($validated);

});
```

The validation rules ensure that the name is present and the email is valid.

If validation fails, Laravel automatically returns the appropriate validation response for the request type.

For larger applications, place this logic in a controller rather than a route closure.

11. Route debugging

Laravel provides useful Artisan commands for inspecting routes.

List all registered routes:

```
php artisan route:list
```

Filter routes by path:

```
php artisan route:list --path=students
```

Show routes using a particular HTTP method:

```
php artisan route:list --method=GET
```

Clear the route cache during development:

```
php artisan route:clear
```

Cache routes for production deployment:

```
php artisan route:cache
```

Route caching improves route registration performance, but it is not a substitute for application optimization.

12. Complete practical project: Student management

This practical combines MVC, GET and POST routes, parameters, named routes, validation, and Blade views.

Step 1: Create the controller

```
php artisan make:controller StudentController
```

Step 2: Add the controller methods

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class StudentController extends Controller
{
    public function index()
    {
        return view('students.index');
    }

    public function create()
    {
        return view('students.create');
    }

    public function show($id)
    {
        return "Student ID: $id";
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email',
        ]);

        return "Student registered: " . $validated['name'];
    }
}
```

Step 3: Define the routes

```
use App\Http\Controllers\StudentController;
use Illuminate\Support\Facades\Route;

Route::get('/students', [
    StudentController::class, 'index'
])->name('students.index');

Route::get('/students/create', [
    StudentController::class, 'create'
])->name('students.create');

Route::post('/students', [
    StudentController::class, 'store'
])->name('students.store');

Route::get('/students/{id}', [
    StudentController::class, 'show'
])->name('students.show');
```

Step 4: Create the student list view

resources/views/students/index.blade.php

```
<h1>Student Management</h1>

<a href="{{ route('students.create') }}">
    Add New Student
</a>

<br><br>

<a href="{{ route('students.show', ['id' => 1]) }}">
    View Student 1
</a>
```

Step 5: Create the registration view

resources/views/students/create.blade.php

```
<h1>Register Student</h1>

<form action="{{ route('students.store') }}" method="POST">

    @csrf

    <label>Name:</label>
    <input type="text" name="name" required>

    <br><br>

    <label>Email:</label>
    <input type="email" name="email" required>

    <br><br>

    <button type="submit">Save Student</button>

</form>
```

Step 6: Run and test

```
php artisan serve
```

Visit /students and click the registration link. Submit the form and check the response.

This example validates the form data but does not save student records to a database. A complete database-backed implementation would use a Student model and Eloquent to create and retrieve records.

13. Revision questions

1. What is routing in Laravel?
2. Where are web routes defined?
3. What is the difference between GET and POST?
4. What are route parameters?
5. What is the purpose of named routes?
6. What is middleware?
7. What is a resource route?
8. Why is CSRF protection important?
9. What does php artisan route:list do?
10. Why should input be validated before it is processed?

14. Practical assignment

Build a Laravel application with the following features:

- A product listing page.
- A form to add a product.
- A route to display a product by ID.
- A POST route to validate submitted product details.
- Named routes for navigation.
- A resource controller to handle CRUD operations.

The application should use Blade views for presentation and a controller to handle the request logic.

The goal is to understand how Laravel connects HTTP requests, routes, controllers, views, and models into a working web application.